

Software Engineering Assignment #3

AI Analysis

Access and Prompt Method of AI Used

For this assignment, I used **Gemini** and **ChatGPT**. Both models I accessed using their **website versions**. For Gemini, I used its **3.1 Pro model with extended thinking** for my prompt. And for ChatGPT, I toggled its **think mode**. Both models were accessed using a free account. The prompt I gave both AIs follows:

Program Overview

The program is a C++ program that takes in a file that has a list of commands. The program will handle these commands and print the output to the user's console.

Implementation

The program will define its own MaxHeap. The MaxHeap must have a list-based implementation that creates a priority queue. No pre-existing heap functions or modules can be used. This code must use object-oriented programming instead of function-oriented programming.

Email Definition and Priority

The MaxHeap should expect to take in an Email. An Email is an object with a category, subject, and date. The category field can only be "Boss", "Subordinate", "Peer", "ImportantPerson", and "OtherPerson". The MaxHeap should first try and sort the email by these categories with "Boss" being the most prioritized and "OtherPerson" being the least. In the event of a conflict, the date field should be used to place the email. The date field is defined in the format "MM-DD-YYYY" and is a string. The newest email will be prioritized over the older email in the event that they share the same category. The subject field is a string.

Commands

There are 4 commands that the program must handle.

EMAIL <sender_category>,<subject_line>,<date>	The email command will add an email to the MaxHeap adhering to the priority information in the previous section.
NEXT	Displays information for next email in MaxHeap in format: "Sender: Subject: Date:"
READ	Removes email from MaxHeap
COUNT	Displays count of unread emails

Input

The program assumes that the format of the test file adheres to the format declared above but does not assume that the commands are not nonsensical. There are a few rules that the program must follow:

- NEXT or READ are invalid if there are no emails in the MaxHeap
- NEXT is invalid without a READ command between
- READ is invalid without a NEXT command between

In the case of errors, no message should be printed, and the bad line is simply ignored.

Example Run

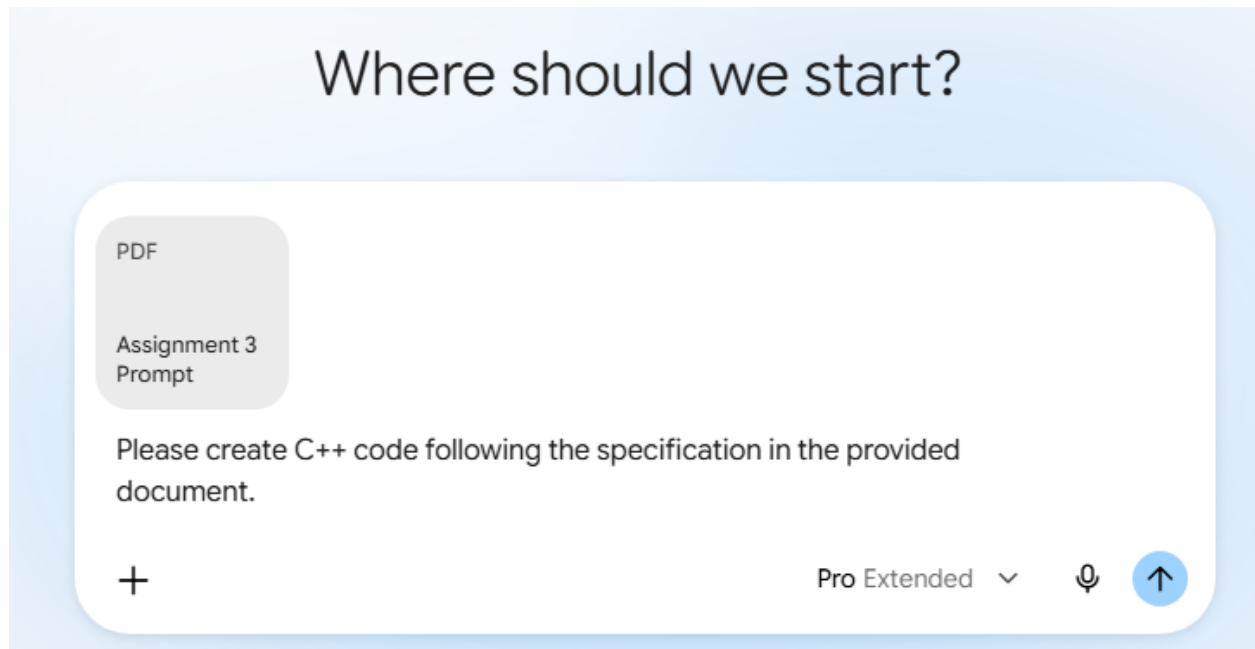
- Sample Test File:
EMAIL Peer,Can you help me on this?,12-01-2024
EMAIL OtherPerson,Try our product,12-19-2024
EMAIL Boss,Important,12-20-2024
EMAIL Subordinate,How do I handle this?,12-25-2024
EMAIL ImportantPerson,Health Insurance Enrollment,12-31-2024
EMAIL Boss,Never Mind,01-03-2025
COUNT
NEXT
READ
NEXT
READ
COUNT
- Output of Sample Test File:

There are 6 emails to read.

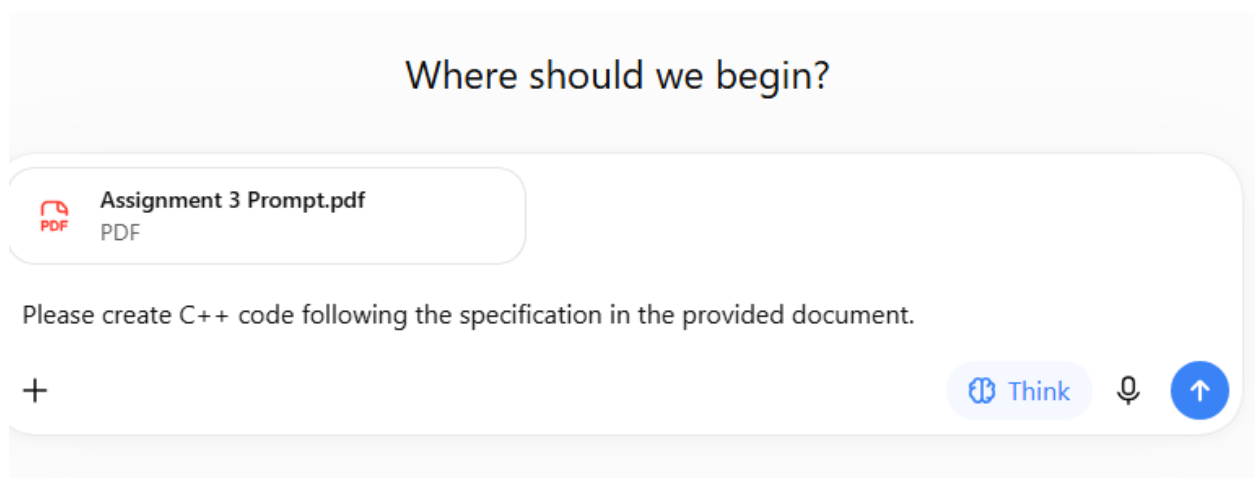
Next email:
Sender: Boss
Subject: Never Mind
Date: 01-03-2025

Next email:
Sender: Boss
Subject: Important
Date: 12-20-2024

There are 4 emails to read.



(Fig. 1 – Gemini Setup)



(Fig 2. - ChatGPT Setup)

AI Analysis

Gemini Output

```
#include <iostream>
#include <vector>
#include <string>
#include <fstream>

using namespace std;

// Helper function to remove leading and trailing whitespace from strings
string trim(const string& str) {
    size_t first = str.find_first_not_of(" \t\r\n");
    if (string::npos == first) return "";
    size_t last = str.find_last_not_of(" \t\r\n");
    return str.substr(first, (last - first + 1));
}

// Class to represent an Email object
class Email {
public:
    string category;
    string subject;
    string date;
    int priority;

    Email(string c, string s, string d) : category(c), subject(s), date(d) {
        // Remove spaces to cleanly check against allowed categories
        string cleanCat = "";
        for (char ch : c) {
            if (ch != ' ') cleanCat += ch;
        }

        // Assign priorities based on the specified hierarchy
        if (cleanCat == "Boss") priority = 5;
        else if (cleanCat == "ImportantPerson") priority = 4;
        else if (cleanCat == "Peer") priority = 3;
        else if (cleanCat == "Subordinate") priority = 2;
        else if (cleanCat == "OtherPerson") priority = 1;
        else priority = 0; // Fallback
    }

    // Overload the less-than operator to define priority for the MaxHeap
```

```

bool operator<(const Email& other) const {
    if (priority != other.priority) {
        return priority < other.priority;
    }

    // Tie-breaker: Date in MM-DD-YYYY format
    int m1 = stoi(date.substr(0, 2));
    int d1 = stoi(date.substr(3, 2));
    int y1 = stoi(date.substr(6, 4));

    int m2 = stoi(other.date.substr(0, 2));
    int d2 = stoi(other.date.substr(3, 2));
    int y2 = stoi(other.date.substr(6, 4));

    // Prioritize newer years, then months, then days
    if (y1 != y2) return y1 < y2;
    if (m1 != m2) return m1 < m2;
    return d1 < d2;
}
};

// Custom MaxHeap class for priority queue implementation
class MaxHeap {
private:
    vector<Email> heap;

    // Restores the max-heap property by bubbling up
    void heapifyUp(int index) {
        while (index > 0) {
            int parent = (index - 1) / 2;
            if (heap[parent] < heap[index]) {
                swap(heap[parent], heap[index]);
                index = parent;
            } else {
                break;
            }
        }
    }

    // Restores the max-heap property by bubbling down
    void heapifyDown(int index) {
        int size = heap.size();
        while (true) {
            int left = 2 * index + 1;

```

```

        int right = 2 * index + 2;
        int largest = index;

        if (left < size && heap[largest] < heap[left]) {
            largest = left;
        }
        if (right < size && heap[largest] < heap[right]) {
            largest = right;
        }

        if (largest != index) {
            swap(heap[index], heap[largest]);
            index = largest;
        } else {
            break;
        }
    }
}

public:
    void insert(const Email& email) {
        heap.push_back(email);
        heapifyUp(heap.size() - 1);
    }

    Email getMax() const {
        return heap[0];
    }

    void extractMax() {
        if (heap.empty()) return;
        heap[0] = heap.back();
        heap.pop_back();
        if (!heap.empty()) {
            heapifyDown(0);
        }
    }

    int getSize() const {
        return heap.size();
    }
};

int main(int argc, char* argv[]) {

```

```

string filename;
if (argc > 1) {
    filename = argv[1];
} else {
    cout << "Enter filename: ";
    cin >> filename;
}

ifstream file(filename);
if (!file.is_open()) {
    return 1;
}

MaxHeap heap;
bool next_called = false;
string line;

while (getline(file, line)) {
    line = trim(line);
    if (line.empty()) continue;

    if (line.substr(0, 6) == "EMAIL ") {
        // Process the EMAIL command
        string rest = line.substr(6);
        size_t comma1 = rest.find(',');
        if (comma1 == string::npos) continue;
        size_t comma2 = rest.find(',', comma1 + 1);
        if (comma2 == string::npos) continue;

        string cat = trim(rest.substr(0, comma1));
        string subj = trim(rest.substr(comma1 + 1, comma2 - comma1 - 1));
        string date = trim(rest.substr(comma2 + 1));

        heap.insert(Email(cat, subj, date));
    }
    else if (line == "NEXT") {
        // NEXT is invalid if no emails or without a READ between it and the last NEXT
        if (heap.getSize() == 0) continue;
        if (next_called) continue;

        Email nextEmail = heap.getMax();
        cout << "Next email:\n";
        cout << "Sender: " << nextEmail.category << "\n";
        cout << "Subject: " << nextEmail.subject << "\n";
    }
}

```

```

        cout << "Date: " << nextEmail.date << "\n";

        next_called = true;
    }
    else if (line == "READ") {
        // READ is invalid if no emails or without a NEXT between it and the last READ
        if (heap.getSize() == 0) continue;
        if (!next_called) continue;

        heap.extractMax();
        next_called = false;
    }
    else if (line == "COUNT") {
        // Display total number of emails in the heap
        cout << "There are " << heap.getSize() << " emails to read.\n";
    }
}

file.close();
return 0;
}

```

ChatGPT

```

#include <iostream>

#include <fstream>

#include <sstream>

#include <string>

#include <vector>

using namespace std;

class Email {

private:

    string category;

    string subject;

    string date;

```

```

public: Email(const string& category, const string& subject, const string& date) :
category(category), subject(subject), date(date) {}

string getCategory() const {
    return category;
}

string getSubject() const {
    return subject;
}

string getDate() const {
    return date;
}

// Converts MM-DD-YYYY into YYYYMMDD so dates can be compared
correctly.
int dateValue() const {
    int month = stoi(date.substr(0, 2));
    int day = stoi(date.substr(3, 2));
    int year = stoi(date.substr(6, 4));

    return year * 10000 + month * 100 + day;
}

};

class MaxHeap{
private:
vector<Email> heap;

int categoryPriority(const string& category) const {
    if (category == "Boss")
        return 5;
    if (category == "Subordinate")
        return 4;
    if (category == "Peer")
        return 3;
    if (category == "ImportantPerson")
        return 2;
    return 1; // OtherPerson
}
}

```

```

// Returns true if a should have higher priority than b.
bool higherPriority(const Email& a, const Email& b) const {
    int aPriority = categoryPriority(a.getCategory());
    int bPriority = categoryPriority(b.getCategory());

    if (aPriority != bPriority) {
        return aPriority > bPriority;
    }

    // If categories are the same, the newest date has priority.
    return a.dateValue() > b.dateValue();
}

void heapifyUp(int index) {
    while (index > 0) {
        int parent = (index - 1) / 2;

        if (!higherPriority(heap[index], heap[parent])) {
            break;
        }

        swap(heap[index], heap[parent]);
        index = parent;
    }
}

void heapifyDown(int index) {
    int size = static_cast<int>(heap.size());

    while (true) {
        int left = 2 * index + 1;
        int right = 2 * index + 2;
        int highest = index;

        if (left < size && higherPriority(heap[left], heap[highest]))
        {
            highest = left;
        }

        if (right < size && higherPriority(heap[right],
heap[highest])) {
            highest = right;
        }

        if (highest == index) {

```

```

        break;
    }

    swap(heap[index], heap[highest]);
    index = highest;
}

public: void insert(const Email& email) { heap.push_back(email);
heapifyUp(static_cast(heap.size()) - 1); }

Email getMax() const {
    return heap[0];
}

void removeMax() {
    if (heap.empty()) {
        return;
    }

    heap[0] = heap.back();
    heap.pop_back();

    if (!heap.empty()) {
        heapifyDown(0);
    }
}

bool empty() const {
    return heap.empty();
}

int size() const {
    return static_cast<int>(heap.size());
}

};

bool validCategory(const string& category) { return category == "Boss" || category ==
"Subordinate" || category == "Peer" || category == "ImportantPerson" || category ==
"OtherPerson"; }

```

```

int main(int argc, char* argv[]) { // The input file is supplied as a command-line argument.
// Example: ./assignment3 input.txt if (argc < 2) { return 0; }

ifstream inputFile(argv[1]);

if (!inputFile.is_open()) {
    return 0;
}

MaxHeap emailQueue;

// true -> NEXT is allowed
// false -> NEXT is not allowed until READ occurs
bool nextAllowed = true;

// true -> READ is allowed because NEXT was most recently performed
// false -> READ is not currently allowed
bool readAllowed = false;

string line;

while (getline(inputFile, line)) {
    // Ignore completely empty lines.
    if (line.empty()) {
        continue;
    }

    // EMAIL command
    if (line.rfind("EMAIL ", 0) == 0) {
        string data = line.substr(6);

        string category;
        string subject;
        string date;

        stringstream ss(data);

        if (!getline(ss, category, ',')) {
            continue;
        }

        if (!getline(ss, subject, ',')) {
            continue;
        }
    }
}

```

```

        if (!getline(ss, date)) {
            continue;
        }

        if (!validCategory(category)) {
            continue;
        }

        Email email(category, subject, date);
        emailQueue.insert(email);

        // EMAIL does not change whether NEXT/READ is required.
    }

    // NEXT command
    else if (line == "NEXT") {
        // NEXT is invalid when there are no emails or when the
        // previous NEXT has not been followed by READ.
        if (emailQueue.empty() || !nextAllowed) {
            continue;
        }

        const Email& email = emailQueue.getMax();

        cout << "Next email:" << endl;
        cout << "Sender: " << email.getCategory() << endl;
        cout << "Subject: " << email.getSubject() << endl;
        cout << "Date: " << email.getDate() << endl;

        nextAllowed = false;
        readAllowed = true;
    }

    // READ command
    else if (line == "READ") {
        // READ is invalid when there are no emails or when NEXT
        // has not been performed since the previous READ.
        if (emailQueue.empty() || !readAllowed) {
            continue;
        }

        emailQueue.removeMax();

        nextAllowed = true;
        readAllowed = false;
    }

```

```

    }

    // COUNT command
    else if (line == "COUNT") {
        cout << "There are " << emailQueue.size()
            << " emails to read." << endl;
    }

    // Any other command is simply ignored.
}

inputFile.close();

return 0;

}

```

Correctness

Email Class

- Both classes have string values for category, date, and subject but Gemini also gives its email class an int priority that represents the priority of its category
- Gemini has public properties while ChatGPT have private properties that have get functions
- Gemini does its comparison by overloading the less-than operator while ChatGPT has no comparison function in its Email class
- ChatGPT offers a function to get the date-value of its string date

Heap Class

- Both ChatGPT and Gemini stores their internal list as a vector of Emails.
- Both classes have classic heapify up and down methodology except in comparison. The ChatGPT MaxHeap uses a function HigherPriority that does the comparison of two email elements. Gemini on the other hand just has it MaxHeap use the less-than sign to trigger the overload in the Email class to compare the two emails

- Both classes have heapify up and downs that use enumeration instead of recursion
- The ChatGPT MaxHeap casts its size values to ints before returning while the Gemini MaxHeap does not
- Both MaxHeaps have exact same getMax() function
- Gemini MaxHeap and ChatGPT MaxHeap both have extract functionality but Gemini's is called removeMax instead of extract.
- ChatGPT's MaxHeap offers an empty function that returns if the MaxHeap is empty or not

Input Handling

- ChatGPT / Gemini handles reading with no emails
- ChatGPT / Gemini fails next, then higher priority email, then read input
- ChatGPT / Gemini fails to decide between technically identical emails
- ChatGPT / Gemini has bad format
- ChatGPT / Gemini handles given input
- ChatGPT / Gemini handles both next and read spam
- ChatGPT has zero message for no input while Gemini starts a dialog that either gets a valid file or exits the program

Correctness

The correctness of both programs is good on paper, but harbors flaws when inspected closely. Technically, the programs are sound. The requirements wanted a list-based heap implementation and object-oriented programming. Both programs adhere to those requirements. It is in execution that the flaws show correctness. The only place that the programs differ in their execution is handling empty input. Both support the idea that the user calls the executable with an argument that is a path to a text file. While good input is assumed in the file itself, good input is not assumed in the actual input to the program. If no argument is provided, ChatGPT simply terminates and tells the user nothing. When testing, I had no idea that the program even ran. On the other hand, Gemini created a text prompt for a path to a file. This is better since it indicates to the user that the program is running and needs a file path. The only issue with this dialog is that it will end if the program cannot find a file path. Ideally, the program should continually prompt the user for a valid file path until one is provided, or the user clearly indicates that they wish the program to end. This would make it so that the user is aware when the program is handling a file or exiting at the user's request. Since the programs are the same in the other areas of correctness, I will not differentiate between them for the rest of this section. Firstly, the programs fail to properly format their output. The output lacks space between the lines and tabs on the next email commands. This means that every input will create an incorrect

result even if it is only slightly so. Besides this wrinkle, the programs can handle basic inputs but fail with specific edge-case situations that nonetheless need to have proper output. On inputs with multiple nexts and reads in a row, the input should perform their functions unless the queue is empty. The programs add a check that a read does not follow a next and vice versa that is completely unnecessary. Besides this, the programs do not indicate that the next and read commands are being called on the empty inbox. Another edge-case failure occurs in the case that emails have the same date and the same sender. The specification states that the priority should be based on first the sender and then the more recent email. Logically, in the case of an email from senders of equal importance and on the same date, the email that was received by the system first would be selected first. Both programs lack this kind of protection. For example, emails with equal senders and dates 1, 2, 3, 4, 5, and 6 enter the system. Currently the programs order them as 1, 6, 5, 4, 3, 2 when the ideal would be 6, 5, 4, 3, 2, 1 since that would be the email in the order of how recent they came in. Overall, while handling good input, the programs have serious issues with their correctness. Both programs have near identical input handling, so Gemini is slightly better at its correctness due to its better answer when the program is called without a file.

Execution Time

The execution time of both programs is similar in the big picture but has some differences in the details. For major algorithms like heapify up and heapify down, both programs have almost the same implementation of algorithms that execute with $O(\log_2 n)$. However, there are still some big differences in how the programs handle their execution time. The most notable difference being in comparison. As previously mentioned, the heapify up and heapify down have a time complexity of $O(\log_2 n)$. However, each of these runs must compare Emails. There are two comparisons in heapify down and one in heapify up for both programs. Therefore, the speed in which these comparisons can be performed is of critical importance for execution time. In this situation, Gemini has an edge over ChatGPT as it already computes the category integer and stores it in the email object. This means that instead of doing what ChatGPT does, which is recomputing the category priority for every single comparison, Gemini only needs to compare the two previously computed values. While these computations are constant, the difference in turning multiple lines into 1 becomes pronounced when this comparison is being called potentially $2\log_2 n$ times and n could be in the millions. So, the execution time is slower in ChatGPT when compared to Gemini.

Space Complexity

Comparing the space complexity of these two programs is very similar to how execution time had to be compared. The broad strokes of each program are similar, but seemingly small differences can cause big impacts. The source of potentially biggest space complexity difference, how each program internally stores the email in the heap, is the same. Both programs use a `vector<Email>` property to store the list. Since this is a dynamic list that expands with the number of emails the system is trying to store, a vector of the email class is the best way to internally store the emails objects in the heap. However, it is in this that the difference between the two programs rears its head. Gemini stores an extra priority integer in its Email object that ChatGPT does not. With same sized heaps, Gemini's heap will need more memory allocation than ChatGPT's because the email object it is storing requires more memory. Therefore, the space complexity of Gemini's program is higher than ChatGPT's program.

Maintainability

Each program is unsatisfactory in its own way in how it defines the Email class. Gemini's public properties for its category, date, and sender means that any section of code can alter these properties when there is no need. Additionally, Gemini's extra property of priority is incredibly dangerous as it is an internal comparison number that has no business being accessible or modifiable by outside functions. ChatGPT has far better property management in this regard as it has private properties for its category, date, and sender values. While the idea of storing priority data instead of recalculating it every time is interesting, the current public nature of the property makes it dangerous to maintainability. ChatGPT hides the properties of the Email class behind public get functions that ensure that they are not being changed outside of the class. If the requirements did change to needing to modify emails, then this could be easily done. The get functions could be removed, and the properties turned public, or set function could be added. This makes private properties much better for maintainability as it prevents developers from erroneously trying to change one of the email values outside of the email class. Both programs lack suitable comments. Neither of the programs have prolog comments, and they have few single line comments that are given to describe massive blocks of code. ChatGPT has an especially bad maintainability due to its heap class having the email comparison function. Were someone looking to change how emails compare, they would most likely expect to find the comparator function inside of the email class somewhere. By having it outside, future developers would need to go on a wild goose chase to figure out where the comparison is being called in the program. Both programs have bad

maintainability due to their lack of comments and out-of-place functions or too accessible properties. Their maintainability is equally bad.

Choice of Template

The choice was very tight between the two programs in this assignment. Both of them struggled in maintainability and correctness. Gemini had better execution time due to its storing of priority while ChatGPT had better space complexity due to computing it on the fly. In the end, ChatGPT's program is the superior pick. Since both programs were close on maintainability and correctness, the decision came down to execution time versus space complexity. Logically, the most important facet for an email program is that it needs to be able to store a massive quantity of emails. Space complexity is the most important value for this particular problem. Therefore, ChatGPT became the pick because it was the program with better space complexity.

Change List

This section will list every change I made to the final program and the reason for the change.

- Priority comparison
 - Reworks how priority is computed by overriding the greater than operator in the Email class to properly compare between emails inside the email class instead of having comparisons in the heap. This also adds an id property to each email to compare which was more recent on emails sent on the same date.
 - Fixes the correctness of the program by allowing emails that are more recent but have the same date to still be properly sorted in the MaxHeap by adding an id property to the Email class.
 - Decreases execution time of program since the new comparison compute date comparisons one at a time which means values of less important date components won't be computed unless their more important counterpart is evaluated and determined to be equal. (Month doesn't compute until it is clear year is equal).
 - Increase maintainability by properly putting the comparison function in the Email class so that it is more obvious to incoming developers where the comparisons are happening
- Complex input
 - Informs user of incorrect usage on more than one argument, continues program until the program is provided with a proper path, or the user indicates a desire to exit.
 - Improves correctness by handling complex user input and either printing an informative error message, processing the indicated file, or exiting the user's request.
- Output fixes
 - Program now allows for nexts and reads to be done multiple times in a row, prints a message when next or read is called on an empty queue, and properly formats output.
 - The next and read changes improve correctness by properly handling edge-case situations without errors and informing the user.
 - The format changes improve correctness since they mean that all output is formatted to the requirements stated in the instructions
- Adding comments

- These comments improve maintainability by clearly indicating what each line does and other important information for the developer reading the comment.